🧩 1. What Are Searching and Sorting Algorithms?

- **Searching algorithms**: used to **find** an element (key) within a data structure (like a list or array).
- **Sorting algorithms**: used to **arrange** data in a particular order (ascending or descending).

They are the foundation of efficient data processing, databases, and computer memory management.

🔍 SEARCHING ALGORITHMS

Let's go step by step 👇

1️⃣ Linear Search (Sequential Search)

🧠 **Concept:**

Search each element one by one until you find the target or reach the end.

⏱️ **Time Complexity:**

- **Best Case:** O(1) — when the element is at the first position.
- **Worst Case:** O(n) — when the element is not present or last in the list.

🧱 **Example:**

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i  # index found
    return -1  # not found


nums = [10, 20, 30, 40, 50]
print(linear_search(nums, 30))  # Output: 2
```

✅ **Use Case:** When data is **unsorted** or small in size.

2️⃣ Binary Search

🧠 Concept:

Efficiently searches **sorted** data by repeatedly dividing the search interval in half.

🔹 Steps:

1. Find middle element.
2. If target < mid → search in left half.
3. If target > mid → search in right half.
4. Repeat until found or range becomes empty.

⏱️ Time Complexity:

- **Best Case:** O(1)
- **Average/Worst Case:** O(log n)

🧮 Example:

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1


nums = [10, 20, 30, 40, 50]
```

```
print(binary_search(nums, 40))  # Output: 3
```

✅ **Use Case:** When data is **sorted** and **large**.

🔃 SORTING ALGORITHMS

Sorting improves data readability and is often a **pre-step for searching** (especially binary search).

1️⃣ Bubble Sort

🧠 **Concept:**

Repeatedly compares **adjacent elements** and swaps them if they're in the wrong order.

⏱️ **Time Complexity:**

- **Best Case:** O(n) (when already sorted)
- **Worst Case:** $O(n^2)$

🧱 **Example:**

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr


nums = [5, 1, 4, 2, 8]
print(bubble_sort(nums))  # [1, 2, 4, 5, 8]
```

✅ **Use Case:** Small datasets or for educational understanding.

2️⃣ Selection Sort

🧠 Concept:

Repeatedly finds the **minimum element** from the unsorted part and places it at the beginning.

⏱️ Time Complexity: O(n²)

🧮 Example:

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr


nums = [64, 25, 12, 22, 11]
print(selection_sort(nums))  # [11, 12, 22, 25, 64]
```

✅ **Use Case:** Simple to implement, used for small arrays.

3️⃣ Insertion Sort

🧠 Concept:

Builds the final sorted array **one item at a time**, inserting each element in its correct position.

⏱️ Time Complexity:

- **Best:** O(n)
- **Worst:** O(n²)

🧮 Example:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

nums = [12, 11, 13, 5, 6]
print(insertion_sort(nums))  # [5, 6, 11, 12, 13]
```

✅ **Use Case:** Best for **nearly sorted data**.

🔽 **Merge Sort (Divide and Conquer)**

🧠 **Concept:**

Divides the array into halves, sorts them, and merges the sorted halves.

⏱️ **Time Complexity: O(n log n)**

🧮 **Example:**

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
```

```
        merge_sort(R)

        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1
        while i < len(L):
            arr[k] = L[i]; i += 1; k += 1
        while j < len(R):
            arr[k] = R[j]; j += 1; k += 1
    return arr

nums = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(nums))
```

✅ **Use Case:** Large datasets, stable sorting.

5️⃣ **Quick Sort (Divide and Conquer)**

🧠 **Concept:**

Selects a **pivot** and partitions the array so that elements smaller than pivot come before it, and greater come after.

⏱️ **Time Complexity:**

- **Average:** O(n log n)
- **Worst:** O(n^2) (when pivot is poorly chosen)

🧱 **Example:**

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)


nums = [3,6,8,10,1,2,1]
print(quick_sort(nums))
```

✅ **Use Case:** Best for general-purpose sorting, used internally by Python's sort().

📊 **COMPARISON TABLE**

Algorithm	Type	Time Complexity	Space	Data Required	Use Case
Linear Search	Searching	O(n)	O(1)	Unsorted	Small data
Binary Search	Searching	O(log n)	O(1)	Sorted	Fast search
Bubble Sort	Sorting	O(n^2)	O(1)	Any	Teaching/demo
Selection Sort	Sorting	O(n^2)	O(1)	Any	Simple sort
Insertion Sort	Sorting	O(n^2)	O(1)	Nearly sorted	Adaptive sort
Merge Sort	Sorting	O(n log n)	O(n)	Any	Large data

Algorithm	Type	Time Complexity	Space	Data Required	Use Case
Quick Sort	Sorting	O(n log n)	O(log n)	Any	General-purpose sort

---

🧠 **Summary Tips**

- For **unsorted search** → use **Linear Search**.
- For **sorted search** → use **Binary Search**.
- For **teaching or small data** → use **Bubble / Selection Sort**.
- For **real-world or large data** → use **Merge Sort** or **Quick Sort**.
- Python's built-in sort() and sorted() use a highly optimized **Timsort**, combining Merge + Insertion sort.